

Geometry

April 13, 2026

0.1 Notes

- This notebook covers the notation commonly encountered in geometry, including fundamental objects (points, lines, angles, triangles), coordinate systems, and differential geometry.
- The notebooks are largely self-contained. If a symbol appears, an explanation will be present at some point in the same notebook. Only if a symbol cannot be covered in this notebook will a reference to another notebook be provided.
- **GitHub does a poor job of rendering these notebooks.** The online render is missing links, symbols are absent, and notations are badly formatted. Clone a local copy or download the notebook and open it locally, or refer to the PDFs in the PDFs/ folder.
- All code uses only Python's standard library.
- **See the Linear_algebra notebook for more on vectors and norms.**

1 Geometry

1.1 Contents

- Fundamentals
 - Points
 - Euclidean space
 - Lines, rays, and segments
 - Segment length
 - Angles
 - Triangles and law of cosines
 - Congruence
 - Similarity
 - Parallel
 - Perpendicular
- Coordinates
 - Cartesian coordinates
 - Polar coordinates
 - Spherical coordinates
 - Cylindrical coordinates
- Differential Geometry
 - Curves and parametrization
 - Tangent vector
 - Normal vector
 - Binormal vector
 - Curvature

- Torsion
- Surfaces
- Euler characteristic
- Gaussian curvature

1.2 Importing Libraries

```
[1]: import math
import random
```

Fundamentals

Points

The most basic geometric object is a point. Points are often denoted with upper case letters such as A , B , C . In the plane, a point can be represented as a 2-tuple:

$$A = (x, y)$$

Points may also be represented as vectors with lower case bold letters $\mathbf{x}$, $\mathbf{y}$, etc.

For more on vectors and tuples, see the `Linear_algebra` and `Collections` notebooks.

```
[2]: A = (3, 4)
B = (7, 1)

print('Point A:', A)
print('Point B:', B)
```

Point A: (3, 4)

Point B: (7, 1)

Euclidean space

The entire Euclidean plane is denoted $\mathbb{R}^2$. More generally, n -dimensional Euclidean space is denoted:

$$\mathbb{R}^n$$

Some authors write E^n or $\mathbb{E}^n$ for Euclidean space to distinguish it from $\mathbb{R}^n$ as a bare set of tuples.

A point in $\mathbb{R}^n$ is an n -tuple $(x_1, x_2, \dots, x_n)$. In two dimensions the coordinates are typically called x and y ; in three dimensions x , y , and z .

```
[3]: # A point in R^2
p_2d = (3.0, 4.0)

# A point in R^3
p_3d = (1.0, 2.0, 5.0)
```

```
print('Point in R^2:', p_2d, ', dimension:', len(p_2d))
print('Point in R^3:', p_3d, ', dimension:', len(p_3d))
```

```
Point in R^2: (3.0, 4.0) , dimension: 2
Point in R^3: (1.0, 2.0, 5.0) , dimension: 3
```

Lines, rays, and segments

Two distinct points A and B determine a unique line which is denoted $\overleftrightarrow{AB}$.

The ray emanating from A and including B is denoted $\overrightarrow{AB}$.

The line segment joining A and B is denoted $\overline{AB}$. For simplicity, some authors write just AB for the segment.

A line segment can be parametrized as the set of all points between A and B :

$$\overline{AB} = \{A + t(B - A) \mid t \in [0, 1]\}$$

```
[4]: A = (1, 2)
     B = (5, 6)

     # Points along the segment AB, parametrized by t in [0, 1]
     for t in [0, 0.25, 0.5, 0.75, 1.0]:
         point = (A[0] + t * (B[0] - A[0]), A[1] + t * (B[1] - A[1]))
         print(f't = {t:.2f} -> point = {point}')
```

```
t = 0.00 -> point = (1, 2)
t = 0.25 -> point = (2.0, 3.0)
t = 0.50 -> point = (3.0, 4.0)
t = 0.75 -> point = (4.0, 5.0)
t = 1.00 -> point = (5.0, 6.0)
```

Segment length

The length of a line segment $\overline{AB}$ is denoted using absolute value bars:

$$|AB|$$

For points $A = (a_1, a_2)$ and $B = (b_1, b_2)$ in $\mathbb{R}^2$, the length is the Euclidean distance:

$$|AB| = \sqrt{(b_1 - a_1)^2 + (b_2 - a_2)^2}$$

More generally, in $\mathbb{R}^n$:

$$|AB| = \sqrt{\sum_{i=1}^n (b_i - a_i)^2}$$

```
[5]: A = (1, 2)
      B = (4, 6)

      length_AB = math.sqrt(sum((b - a)**2 for a, b in zip(A, B)))
      print('|AB| =', length_AB)
```

|AB| = 5.0

Angles

An angle is the union of two rays emanating from the same point. If the rays are $\overrightarrow{AB}$ and $\overrightarrow{AC}$, the angle can be denoted:

$$\angle BAC \quad \text{or} \quad \angle A$$

The measure of an angle is written $m\angle BAC$. Some authors omit m and use the same notation for an angle and its measure.

Angles are typically measured in radians or degrees. A full rotation is 2π radians or 360° .

The angle between two vectors $\mathbf{u}$ and $\mathbf{v}$ originating from the same point can be found using the dot product:

$$\cos(\angle(\mathbf{u}, \mathbf{v})) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

```
[6]: A = (0, 0)
      B = (1, 0)
      C = (0, 1)

      # Vectors from A to B and A to C
      u = (B[0] - A[0], B[1] - A[1])
      v = (C[0] - A[0], C[1] - A[1])

      dot = u[0]*v[0] + u[1]*v[1]
      mag_u = math.sqrt(u[0]**2 + u[1]**2)
      mag_v = math.sqrt(v[0]**2 + v[1]**2)

      angle_rad = math.acos(dot / (mag_u * mag_v))
      print('Angle BAC:', angle_rad, 'radians')
      print('Angle BAC:', math.degrees(angle_rad), 'degrees')
```

Angle BAC: 1.5707963267948966 radians

Angle BAC: 90.0 degrees

Triangles and law of cosines

A triangle is the union of three line segments determined by three points and is denoted:

$$\triangle ABC$$

In $\triangle ABC$, the angles are named by their vertex points ($\angle A$, $\angle B$, $\angle C$) and the lengths of the sides are named by lower case letters corresponding to the opposite angle. That is, a is the length of the segment opposite $\angle A$ (i.e., $|BC|$), $b = |AC|$, and $c = |AB|$.

The *law of cosines* relates the side lengths and an angle:

$$c^2 = a^2 + b^2 - 2ab \cos C$$

Here c is the length of the segment $\overline{AB}$ and $\cos C$ is the cosine of the measure of $\angle C$.

```
[7]: # Triangle with vertices A, B, C
A = (0, 0)
B = (4, 0)
C = (1, 3)

a = math.dist(B, C) # side opposite angle A
b = math.dist(A, C) # side opposite angle B
c = math.dist(A, B) # side opposite angle C

# Angle C using law of cosines: cos C = (a^2 + b^2 - c^2) / (2ab)
cos_C = (a**2 + b**2 - c**2) / (2 * a * b)
angle_C = math.acos(cos_C)

print('Side a:', round(a, 4), ', b:', round(b, 4), ', c:', round(c, 4))
print('Angle C:', round(math.degrees(angle_C), 4), 'degrees')

# Verify law of cosines: c^2 = a^2 + b^2 - 2ab cos C
print('c^2 =', c**2, ', a^2 + b^2 - 2ab cos C =', round(a**2 + b**2 -
↪ 2*a*b*cos_C, 4))
```

Side a: 4.2426 , b: 3.1623 , c: 4.0

Angle C: 63.4349 degrees

$c^2 = 16.0$, $a^2 + b^2 - 2ab \cos C = 16.0$

Congruence

Two geometric figures are *congruent* if there is an isometry (distance-preserving transformation) of space that maps one to the other. Congruence is denoted:

$$\cong$$

For example, $\angle A \cong \angle Y$ means the two angles have the same measure, and $\triangle ABC \cong \triangle DEF$ means the two triangles are congruent.

```
[8]: # Two triangles are congruent if all side lengths match
triangle_1 = {'a': 3, 'b': 4, 'c': 5}
triangle_2 = {'a': 3, 'b': 4, 'c': 5}
triangle_3 = {'a': 3, 'b': 4, 'c': 6}

def is_congruent(t1, t2):
    return sorted(t1.values()) == sorted(t2.values())

print('Triangle 1   Triangle 2:', is_congruent(triangle_1, triangle_2))
print('Triangle 1   Triangle 3:', is_congruent(triangle_1, triangle_3))
```

```
Triangle 1   Triangle 2: True
Triangle 1   Triangle 3: False
```

Similarity

Two geometric figures are *similar* if one can be obtained from the other by scaling (and possibly rotation or reflection). Similarity is denoted:

$\sim$

For example, $\triangle ABC \sim \triangle DEF$ means the two triangles have the same angles and their corresponding side lengths are proportional:

$$\frac{a}{d} = \frac{b}{e} = \frac{c}{f}$$

```
[9]: # Two triangles are similar if side length ratios are equal
t1 = [3, 4, 5]
t2 = [6, 8, 10]
t3 = [3, 4, 6]

def is_similar(s1, s2):
    s1, s2 = sorted(s1), sorted(s2)
    ratios = [a / b for a, b in zip(s1, s2)]
    return all(math.isclose(r, ratios[0]) for r in ratios)

print('Triangle 1 ~ Triangle 2:', is_similar(t1, t2))
print('Triangle 1 ~ Triangle 3:', is_similar(t1, t3))
```

```
Triangle 1 ~ Triangle 2: True
Triangle 1 ~ Triangle 3: False
```

Parallel

Two lines, rays, or segments are *parallel* if they lie in the same plane and do not intersect. This is denoted:

$$\overline{AB} \parallel \overline{XY}$$

Two segments are parallel if and only if their direction vectors are scalar multiples of each other.

```
[10]: A, B = (0, 0), (2, 4)
      X, Y = (1, 0), (3, 4)

      # Direction vectors
      d1 = (B[0] - A[0], B[1] - A[1])
      d2 = (Y[0] - X[0], Y[1] - X[1])

      # Parallel if cross product is zero
      cross = d1[0] * d2[1] - d1[1] * d2[0]
      print('Direction AB:', d1, ', Direction XY:', d2)
      print('AB XY:', cross == 0)
```

```
Direction AB: (2, 4) , Direction XY: (2, 4)
AB XY: True
```

Perpendicular

Two lines, rays, or segments are *perpendicular* if they intersect at a right angle (90° or $\frac{\pi}{2}$ radians). This is denoted:

$$\overline{AB} \perp \overline{CD}$$

Two segments are perpendicular if and only if their direction vectors have a dot product of zero.

```
[11]: A, B = (0, 0), (3, 0)
      C, D = (0, 0), (0, 5)

      d1 = (B[0] - A[0], B[1] - A[1])
      d2 = (D[0] - C[0], D[1] - C[1])

      dot = d1[0] * d2[0] + d1[1] * d2[1]
      print('Direction AB:', d1, ', Direction CD:', d2)
      print('AB CD:', dot == 0)
```

```
Direction AB: (3, 0) , Direction CD: (0, 5)
AB CD: True
```

Coordinates

Cartesian coordinates

In analytic geometry, points are specified by a list of numbers. The most common system is *Cartesian coordinates* (also called *rectangular coordinates*). Points are specified by n -tuples.

In two dimensions, a point is written (x, y) . In three dimensions, (x, y, z) . In higher dimensions, subscripts are used: $(x_1, x_2, \dots, x_n)$.

The letters x and y are traditionally used for Cartesian coordinates in two dimensions, and x, y, z in three dimensions.

```
[12]: # Cartesian coordinates in 2D and 3D
p_2d = (3, 4)
p_3d = (1, 2, 5)

# Distance from origin in each case
dist_2d = math.sqrt(sum(c**2 for c in p_2d))
dist_3d = math.sqrt(sum(c**2 for c in p_3d))

print('2D point:', p_2d, ', distance from origin:', round(dist_2d, 4))
print('3D point:', p_3d, ', distance from origin:', round(dist_3d, 4))
```

```
2D point: (3, 4) , distance from origin: 5.0
3D point: (1, 2, 5) , distance from origin: 5.4772
```

Polar coordinates

In the plane, a point may be specified by a distance r from the origin and an angle θ from the positive x -axis. The point is written (r, θ) .

The conversion from polar to Cartesian coordinates is:

$$x = r \cos \theta, \quad y = r \sin \theta$$

The conversion from Cartesian to polar coordinates is:

$$r = \sqrt{x^2 + y^2}, \quad \theta = \text{atan2}(y, x)$$

```
[13]: # Polar to Cartesian
r = 5
theta = math.pi / 4

x = r * math.cos(theta)
y = r * math.sin(theta)
print(f'Polar ({r}, {round(theta, 4)}) -> Cartesian ({round(x, 4)}, {round(y, 4)})')
```



```
# Cartesian back to Polar
r_back = math.sqrt(x**2 + y**2)
theta_back = math.atan2(y, x)
print(f'Cartesian ({round(x, 4)}, {round(y, 4)}) -> Polar ({round(r_back, 4)}, {round(theta_back, 4)})')
```

Polar (5, 0.7854) -> Cartesian (3.5355, 3.5355)
 Cartesian (3.5355, 3.5355) -> Polar (5.0, 0.7854)

Spherical coordinates

Spherical coordinates are a natural extension of polar coordinates to three dimensions. Each point is specified by a distance r from the origin, a rotation θ clockwise from the $+x$ -axis in the xy -plane, and a second rotation ϕ out of the xy -plane.

The point is written (r, θ, ϕ) where θ is called the *azimuth* and ϕ is called the *inclination* or *polar angle*.

The conversion from spherical to Cartesian coordinates is:

$$x = r \cos \theta \sin \phi, \quad y = r \sin \theta \sin \phi, \quad z = r \cos \phi$$

```
[14]: r = 10
theta = math.pi / 4 # azimuth
phi = math.pi / 3   # inclination

x = r * math.cos(theta) * math.sin(phi)
y = r * math.sin(theta) * math.sin(phi)
z = r * math.cos(phi)

print(f'Spherical (r={r}, theta={round(theta, 4)}, phi={round(phi, 4)})')
print(f'Cartesian (x={round(x, 4)}, y={round(y, 4)}, z={round(z, 4)})')

# Verify: x^2 + y^2 + z^2 should equal r^2
print('x^2 + y^2 + z^2 =', round(x**2 + y**2 + z**2, 4), ', r^2 =', r**2)
```

Spherical (r=10, theta=0.7854, phi=1.0472)
 Cartesian (x=6.1237, y=6.1237, z=5.0)
 $x^2 + y^2 + z^2 = 100.0$, $r^2 = 100$

Cylindrical coordinates

Cylindrical coordinates are another extension of polar coordinates to three dimensions. Points in space are located in the x, y -plane using polar coordinates (r, θ) and then a height z above that plane is added. The full specification is the triple:

$$(r, \theta, z)$$

The conversion from cylindrical to Cartesian coordinates is:

$$x = r \cos \theta, \quad y = r \sin \theta, \quad z = z$$

```
[15]: r = 5
theta = math.pi / 6
z = 8

x = r * math.cos(theta)
y = r * math.sin(theta)
# z remains z

print(f'Cylindrical (r={r}, theta={round(theta, 4)}, z={z})')
print(f'Cartesian (x={round(x, 4)}, y={round(y, 4)}, z={z})')
```

Cylindrical (r=5, theta=0.5236, z=8)

Cartesian (x=4.3301, y=2.5, z=8)

Differential Geometry

Curves and parametrization

A *curve* is the image of a continuous function $\alpha : [a, b] \rightarrow \mathbb{R}^k$. The function α is called a *parametrization* of the curve.

A parametrization is *unit speed* if $\|\alpha'(s)\| = 1$ for all $s \in [a, b]$.

A common example is a circle of radius r parametrized by:

$$\alpha(t) = (r \cos t, r \sin t), \quad t \in [0, 2\pi]$$

```
[16]: # Parametrize a circle of radius 3
r = 3
points = []
for i in range(8):
    t = 2 * math.pi * i / 8
    point = (round(r * math.cos(t), 4), round(r * math.sin(t), 4))
    points.append(point)
print(f't = {round(t, 4)} -> alpha(t) = {point}')
```

```
t = 0.0 -> alpha(t) = (3.0, 0.0)
t = 0.7854 -> alpha(t) = (2.1213, 2.1213)
t = 1.5708 -> alpha(t) = (0.0, 3.0)
t = 2.3562 -> alpha(t) = (-2.1213, 2.1213)
t = 3.1416 -> alpha(t) = (-3.0, 0.0)
t = 3.927 -> alpha(t) = (-2.1213, -2.1213)
t = 4.7124 -> alpha(t) = (-0.0, -3.0)
t = 5.4978 -> alpha(t) = (2.1213, -2.1213)
```

Tangent vector

The *tangent vector* to a curve $\alpha(s)$ at parameter s is denoted:

$$\mathbf{T}(s) = \alpha'(s)$$

For a unit speed curve, $\|\mathbf{T}(s)\| = 1$. The tangent vector gives the direction in which the curve is moving at that point.

Numerically, the tangent can be approximated by a finite difference:

$$\mathbf{T}(s) \approx \frac{\alpha(s+h) - \alpha(s)}{h}$$

```
[17]: # Tangent vector to a circle alpha(t) = (cos t, sin t)
def alpha(t):
    return (math.cos(t), math.sin(t))

t = math.pi / 4
h = 1e-8

# Numerical derivative
a0 = alpha(t)
a1 = alpha(t + h)
T = ((a1[0] - a0[0]) / h, (a1[1] - a0[1]) / h)

print(f'alpha({round(t, 4)}) = ({round(a0[0], 4)}, {round(a0[1], 4)})')
print(f'T({round(t, 4)})    ({round(T[0], 4)}, {round(T[1], 4)})')

# Exact: T(t) = (-sin t, cos t)
T_exact = (-math.sin(t), math.cos(t))
print(f'T exact = ({round(T_exact[0], 4)}, {round(T_exact[1], 4)})')

alpha(0.7854) = (0.7071, 0.7071)
T(0.7854)    (-0.7071, 0.7071)
T exact     = (-0.7071, 0.7071)
```

Normal vector

The *normal vector* to a curve $\alpha(s)$ is the unit vector in the direction of $\mathbf{T}'(s)$:

$$\mathbf{N}(s) = \frac{\mathbf{T}'(s)}{\|\mathbf{T}'(s)\|}$$

The normal vector is perpendicular to the tangent vector and points toward the center of curvature.

```
[18]: # Normal vector to a unit circle at t = pi/4
# alpha(t) = (cos t, sin t), T(t) = (-sin t, cos t)
# T'(t) = (-cos t, -sin t)

t = math.pi / 4
T_prime = (-math.cos(t), -math.sin(t))
mag = math.sqrt(T_prime[0]**2 + T_prime[1]**2)
N = (T_prime[0] / mag, T_prime[1] / mag)

print(f"T'({round(t, 4)}) = ({round(T_prime[0], 4)}, {round(T_prime[1], 4)})")
print(f"N({round(t, 4)}) = ({round(N[0], 4)}, {round(N[1], 4)})")

# Verify T · N = 0 (perpendicular)
T_vec = (-math.sin(t), math.cos(t))
dot = T_vec[0] * N[0] + T_vec[1] * N[1]
print('T · N =', round(dot, 10), '(should be 0)')

T'(0.7854) = (-0.7071, -0.7071)
N(0.7854) = (-0.7071, -0.7071)
T · N = 0.0 (should be 0)
```

Binormal vector

For a curve in $\mathbb{R}^3$, the *binormal vector* is defined as the cross product of the tangent and normal vectors:

$$\mathbf{B}(s) = \mathbf{T}(s) \times \mathbf{N}(s)$$

The triple $(\mathbf{T}, \mathbf{N}, \mathbf{B})$ is called the *Frenet-Serret frame* and forms an orthonormal basis at each point of the curve.

For more on the cross product, see the `Linear_algebra` notebook.

```
[19]: # Helix: alpha(t) = (cos t, sin t, t)
# T = (-sin t, cos t, 1) / sqrt(2)
# N = (-cos t, -sin t, 0)
# B = T x N

t = math.pi / 4
s2 = math.sqrt(2)
T = (-math.sin(t)/s2, math.cos(t)/s2, 1/s2)
N = (-math.cos(t), -math.sin(t), 0)

# Cross product T x N
B = (
    T[1]*N[2] - T[2]*N[1],
    T[2]*N[0] - T[0]*N[2],
    T[0]*N[1] - T[1]*N[0])
```

```

    T[0]*N[1] - T[1]*N[0]
)

print(f'T = ({round(T[0],4)}, {round(T[1],4)}, {round(T[2],4)})')
print(f'N = ({round(N[0],4)}, {round(N[1],4)}, {round(N[2],4)})')
print(f'B = T x N = ({round(B[0],4)}, {round(B[1],4)}, {round(B[2],4)})')

T = (-0.5, 0.5, 0.7071)
N = (-0.7071, -0.7071, 0)
B = T x N = (0.5, -0.5, 0.7071)

```

Curvature

The *curvature* of a curve $\alpha(s)$ at parameter s measures how quickly the curve changes direction. For a unit speed curve, the curvature is:

$$\kappa(s) = \|\mathbf{T}'(s)\|$$

A straight line has curvature $\kappa = 0$ everywhere. A circle of radius r has constant curvature $\kappa = \frac{1}{r}$.

```

[20]: # Curvature of a circle of radius r
      # alpha(t) = (r cos t, r sin t), unit speed param: s = t/r
      # kappa = 1/r

      for r in [1, 2, 5, 10]:
          kappa = 1 / r
          print(f'Circle of radius {r:>2}: kappa = 1/{r} = {kappa}')

```

```

Circle of radius  1: kappa = 1/1 = 1.0
Circle of radius  2: kappa = 1/2 = 0.5
Circle of radius  5: kappa = 1/5 = 0.2
Circle of radius 10: kappa = 1/10 = 0.1

```

Torsion

The *torsion* of a space curve $\alpha(s)$ measures how much the curve deviates from being planar. It is defined as:

$$\tau(s) = -\mathbf{B}'(s) \cdot \mathbf{N}(s)$$

A curve that lies entirely in a plane has torsion $\tau = 0$. A circular helix $(\cos t, \sin t, bt)$ has constant torsion $\tau = \frac{b}{1+b^2}$.

```

[21]: # Torsion of a helix alpha(t) = (cos t, sin t, b*t)
      # tau = b / (1 + b^2)

```

```

for b in [0, 0.5, 1, 2]:
    tau = b / (1 + b**2)
    label = 'circle (planar)' if b == 0 else f'helix with b={b}'
    print(f'{label}: tau = {round(tau, 4)}')

```

```

circle (planar): tau = 0.0
helix with b=0.5: tau = 0.4
helix with b=1: tau = 0.5
helix with b=2: tau = 0.4

```

Surfaces

Various geometric surfaces have special notation:

- **Sphere:** The standard sphere in $\mathbb{R}^3$ is S^2 . In general, S^k is a k -dimensional sphere. The circle S^1 is the 1-dimensional sphere.
- **Ball:** The notation $B(\mathbf{x}, r)$ is often used for the ball centered at $\mathbf{x}$ with radius r :

$$B(\mathbf{x}, r) = \{\mathbf{y} : \|\mathbf{y} - \mathbf{x}\| \leq r\}$$

- **Torus:** The surface of a doughnut is denoted $\mathbb{T}^2$, $S^1 \times S^1$, or $\mathbb{R}^2/\mathbb{Z}^2$.
- **Projective plane:** Denoted $\mathbb{RP}^2$.
- **Hyperbolic plane:** Denoted $\mathbb{H}^2$.

```

[22]: # Check if a point y is inside the ball B(x, r)
def in_ball(x, y, r):
    dist = math.sqrt(sum((a - b)**2 for a, b in zip(x, y)))
    return dist <= r

center = (0, 0, 0)
r = 5

test_points = [(1, 2, 2), (3, 4, 0), (3, 4, 5)]
for p in test_points:
    dist = math.sqrt(sum((a - b)**2 for a, b in zip(center, p)))
    print(f'Point {p}, dist = {round(dist, 4)}, in B(0, {r}): {in_ball(center, p, r)}')

```

```

Point (1, 2, 2), dist = 3.0, in B(0, 5): True
Point (3, 4, 0), dist = 5.0, in B(0, 5): True
Point (3, 4, 5), dist = 7.0711, in B(0, 5): False

```

Euler characteristic

The *Euler characteristic* of a surface M is denoted $\chi(M)$. If a connected graph is embedded on the surface of M so that every face is homeomorphic to the interior of a disc, then:

$$\chi(M) = v - e + f$$

where v is the number of vertices, e is the number of edges, and f is the number of faces.

For example, a convex polyhedron (topologically a sphere) always satisfies $\chi = 2$.

```
[23]: # Euler characteristic for common polyhedra (all are spheres, so chi = 2)
polyhedra = {
    'Tetrahedron': {'v': 4, 'e': 6, 'f': 4},
    'Cube':        {'v': 8, 'e': 12, 'f': 6},
    'Octahedron':  {'v': 6, 'e': 12, 'f': 8},
    'Dodecahedron': {'v': 20, 'e': 30, 'f': 12},
    'Icosahedron': {'v': 12, 'e': 30, 'f': 20},
}

for name, p in polyhedra.items():
    chi = p['v'] - p['e'] + p['f']
    print(f"{name:15s}: v={p['v']:>2}, e={p['e']:>2}, f={p['f']:>2}, chi = {chi}")
```

```
Tetrahedron    : v= 4, e= 6, f= 4, chi = 2
Cube           : v= 8, e=12, f= 6, chi = 2
Octahedron     : v= 6, e=12, f= 8, chi = 2
Dodecahedron   : v=20, e=30, f=12, chi = 2
Icosahedron    : v=12, e=30, f=20, chi = 2
```

Gaussian curvature

The *Gaussian curvature* at a point p on a surface is denoted $K(p)$. It is related to the *principal curvatures* k_1 and k_2 of the surface by:

$$K = k_1 k_2$$

The Gauss-Bonnet theorem relates the total Gaussian curvature of a compact, boundaryless surface M to its Euler characteristic (See: Euler characteristic):

$$\int_M K dA = 2\pi\chi(M)$$

For a sphere of radius r , the Gaussian curvature is constant at $K = \frac{1}{r^2}$, and the total curvature over the sphere is 4π , matching $2\pi\chi(S^2) = 2\pi \cdot 2 = 4\pi$.

```
[24]: # Gaussian curvature of a sphere of radius r: K = 1/r^2
# Total curvature = integral of K over surface = K * 4*pi*r^2 = 4*pi
# Should equal 2*pi*chi(S^2) = 2*pi*2 = 4*pi

for r in [1, 2, 5, 10]:
```

```

K = 1 / r**2
surface_area = 4 * math.pi * r**2
total_curvature = K * surface_area
gauss_bonnet = 2 * math.pi * 2 #  $\chi(S^2) = 2$ 
print(f'r={r:>2}: K=1/{r**2:<3}, total curvature = {round(total_curvature, 4)}, 2*pi*chi = {round(gauss_bonnet, 4)}')

```

```

r= 1: K=1/1 , total curvature = 12.5664, 2*pi*chi = 12.5664
r= 2: K=1/4 , total curvature = 12.5664, 2*pi*chi = 12.5664
r= 5: K=1/25 , total curvature = 12.5664, 2*pi*chi = 12.5664
r=10: K=1/100, total curvature = 12.5664, 2*pi*chi = 12.5664

```